

APOSTILA DO PROJETO ALONGUEBE

React, Vite e WordPress Headless para iniciantes

Material de estudo baseado no projeto real do Salão Alonguebe
Atualizado em julho de 2026

=====

COMO USAR ESTA APOSTILA

=====

Este material foi escrito para quem ainda está começando. Você não precisa conhecer React, WordPress ou APIs para acompanhar. Leia os capítulos na ordem na primeira vez. Depois, use o capítulo “Catálogo de arquivos” como consulta.

O objetivo não é decorar código. O objetivo é compreender:

1. qual problema cada parte resolve;
2. como os dados caminham pelo sistema;
3. onde fazer uma alteração sem quebrar outras partes;
4. como reaproveitar a arquitetura em projetos futuros.

Algumas informações do salão ainda são exemplos, como telefone e endereço. Antes de publicar, elas precisam ser substituídas pelos dados verdadeiros.

=====

1. O QUE FOI CONSTRUÍDO

=====

O projeto é o site institucional de um salão de beleza. O visitante pode:

- conhecer o salão e seus diferenciais;
- visualizar serviços, galeria e depoimentos;
- consultar localização e perguntas frequentes;
- solicitar um agendamento;
- enviar uma mensagem;
- iniciar uma conversa pelo WhatsApp.

Existem duas aplicações trabalhando juntas:

FRONTEND

É a parte visual aberta no navegador. Foi criada com React e Vite e fica na pasta client. React organiza a página em componentes. Vite inicia o servidor de desenvolvimento e cria os arquivos otimizados para publicação.

CMS / BACKEND ATUAL

É o WordPress com o plugin Alonguebe Headless CMS. Ele oferece o painel onde a equipe edita conteúdo e armazena mensagens e agendamentos. O WordPress não desenha a página pública; ele fornece dados ao React por uma API.

BACKEND LEGADO

A pasta server contém uma API Express criada numa fase anterior. O frontend atual não envia formulários para ela. Ela foi mantida como referência de estudo e pode ser iniciada separadamente.

=====

2. O QUE SIGNIFICA “WORDPRESS HEADLESS”

=====

Num WordPress tradicional, o próprio tema monta o HTML visto pelo visitante. Num WordPress headless, separamos as responsabilidades:

- WordPress: painel, conteúdo e armazenamento;

- React: interface apresentada ao visitante;
- REST API: ponte usada para transportar os dados entre os dois.

Uma comparação simples:

O WordPress é a cozinha.

O React é o salão do restaurante.

A API é o garçom que leva o pedido e traz o prato.

Essa separação permite criar uma interface moderna sem perder a facilidade do painel WordPress. Em troca, precisamos cuidar da conexão, dos fallbacks e da segurança das rotas públicas.

3. COMO O SITE COMEÇA

O caminho de inicialização é este:

1. O navegador recebe client/index.html.
2. Esse HTML contém uma área vazia chamada div#root.
3. client/src/main.jsx conecta o React a essa área.
4. CmsProvider inicia a consulta ao WordPress.
5. App.jsx monta as seções do site na ordem correta.
6. Cada componente apresenta o conteúdo do CMS ou um texto reserva.

O arquivo HTML é como o terreno. main.jsx entrega o terreno ao React. App.jsx é a planta geral, e cada componente é um cômodo separado.

4. COMO O CONTEÚDO CHEGA À TELA

O frontend usa duas fontes do WordPress.

4.1 API PERSONALIZADA DO PLUGIN

CmsContext.jsx solicita:

GET /wp-json/alonguebe/v1/content

Essa rota devolve, numa única resposta:

- configurações gerais;
- serviços;
- itens da galeria;
- depoimentos;
- perguntas frequentes.

O resultado é armazenado num Context do React. Context é uma forma de deixar uma informação disponível para vários componentes sem precisar passá-la de pai para filho muitas vezes.

4.2 API PADRÃO DE PÁGINAS DO WORDPRESS

O hook useWordPressPage consulta páginas comuns pelo slug. Slug é o nome curto usado na URL. O projeto procura:

- inicio: pode fornecer título, texto e imagem da Hero;
- sobre: pode fornecer título e texto institucional.

4.3 FALLBACK

Fallback é o conteúdo reserva. Se o WordPress estiver desligado, as páginas ainda exibem os textos e arrays escritos nos componentes React.

Isso melhora a resistência do site, mas tem um limite importante: formulários não podem salvar dados sem o WordPress, pois precisam da API.

=====

5. CONCEITOS DE REACT USADOS NO PROJETO

=====

COMPONENTE

Uma função que retorna JSX. JSX parece HTML, mas permite inserir JavaScript entre chaves. Cada seção visual foi isolada em seu próprio componente.

PROP

Um valor enviado por um componente pai para um filho. App envia, por exemplo, o número do WhatsApp aos componentes que precisam criar links.

ESTADO – useState

Memória temporária de um componente. Quando o estado muda, React atualiza a tela. O menu aberto, os campos dos formulários e a pergunta ativa do FAQ são exemplos de estado.

EFEITO – useEffect

Código que sincroniza o componente com algo externo. O projeto o utiliza para buscar dados, observar rolagem e copiar o serviço selecionado para o formulário.

CONTEXT

Compartilha um valor com muitos componentes. CmsContext evita que cada seção tenha de repetir a consulta geral ao plugin.

HOOK PERSONALIZADO

Uma função iniciada por “use” que reaproveita lógica de React. O projeto usa useWordPressPage para centralizar o carregamento de páginas do WordPress.

FORMULÁRIO CONTROLADO

Cada campo recebe seu valor do estado e atualiza o estado em onChange. O React sempre conhece o conteúdo atual do formulário.

RENDERIZAÇÃO CONDICIONAL

Um trecho diferente aparece conforme uma condição. Depois de um agendamento bem-sucedido, por exemplo, o formulário dá lugar à mensagem de sucesso.

MAP

Transforma cada objeto de uma lista em um elemento visual. É usado em serviços, galeria, depoimentos, FAQ e links de navegação.

KEY

Identificador de um item numa lista React. Ajuda o React a saber exatamente qual item mudou, entrou ou saiu.

=====

6. COMO A ESCOLHA DE UM SERVIÇO CHEGA AO FORMULÁRIO

=====

Services e AppointmentForm são componentes irmãos. Um não controla diretamente o estado do outro. Por isso, a informação passa pelo componente pai App:

1. O visitante clica em “Solicitar Horário”.
2. Services chama a função onSelectService recebida por prop.
3. App atualiza o estado selectedService.

4. App envia `selectedService` para `AppointmentForm`.
5. Um `useEffect` copia o novo serviço para `formData.service`.
6. A página rola suavemente até a seção de agendamento.

Esse padrão é conhecido como “elevar o estado”. O dado fica no ancestral comum mais próximo dos componentes que precisam compartilhá-lo.

7. FLUXO DO AGENDAMENTO

O formulário mantém estes campos:

- `name`: nome;
- `email`: e-mail;
- `phone`: telefone ou WhatsApp;
- `service`: serviço desejado;
- `date`: data;
- `time`: horário;
- `notes`: observações.

Ao enviar:

1. `preventDefault` impede o recarregamento padrão da página.
2. `validate` verifica os campos no navegador.
3. `isSubmitting` bloqueia envios repetidos.
4. `fetch` envia JSON para `/wp-json/alonguebe/v1/appointments`.
5. O plugin valida novamente os dados no servidor.
6. O WordPress cria um conteúdo interno do tipo `along_appointment`.
7. Os dados são armazenados como conteúdo e metadados.
8. O status inicial é “Pendente”.
9. A resposta usa HTTP 201, que significa “recurso criado”.
10. O React mostra sucesso e oferece encaminhamento pelo WhatsApp.

Por que validar duas vezes?

A validação no React é rápida e ajuda o visitante. A validação no servidor é obrigatória para segurança, pois alguém pode chamar a API sem usar o site.

8. FLUXO DA MENSAGEM DE CONTATO

Contact usa um fluxo semelhante, porém menor:

1. confere nome, e-mail e mensagem;
2. envia JSON para `/wp-json/alonguebe/v1/messages`;
3. o plugin cria um conteúdo interno `along_message`;
4. a equipe consulta o envio no menu Mensagens do WordPress;
5. o formulário é limpo após sucesso.

Os estados `null`, `loading`, `success` e `error` determinam o que aparece na tela.

9. COMO O PLUGIN WORDPRESS FUNCIONA

O plugin concentra painel, conteúdo e API num arquivo PHP.

9.1 CONFIGURAÇÕES GERAIS

`alonguebe_setting_fields` define os campos editáveis, seus rótulos e padrões.

Todos são salvos na opção `alonguebe_site_settings`. Isso inclui contatos, textos das seções, botões, horários e URLs de imagens da seção Sobre.

9.2 TIPOS PERSONALIZADOS DE CONTEÚDO

O WordPress chama esses tipos de Custom Post Types ou CPTs:

- `along_service`: serviços;
- `along_gallery`: galeria;
- `along_testimonial`: depoimentos;
- `along_faq`: perguntas frequentes;
- `along_appointment`: agendamentos recebidos;
- `along_message`: mensagens recebidas.

Os quatro primeiros são conteúdo editorial. Os dois últimos são caixas de entrada administrativas e não devem formar páginas públicas.

9.3 METADADOS

Metadados são informações extras ligadas a um item. Exemplos:

- `_along_category`: categoria do serviço ou imagem;
- `_along_city`: cidade de um depoimento;
- `_along_stars`: quantidade de estrelas;
- `_along_order`: ordem da lista;
- `_along_phone`: telefone recebido;
- `_along_status`: situação do envio.

O prefixo `_along_` evita conflito com outros plugins.

9.4 SEGURANÇA ADMINISTRATIVA

O `nonce` verifica se um salvamento começou numa tela legítima do WordPress. `current_user_can` verifica se o usuário tem autorização para editar o item. Funções `sanitize_*` limpam os valores antes de salvá-los. Funções `esc_*` protegem os valores antes de exibi-los no HTML do painel.

9.5 ROTAS

GET `/alonguebe/v1/content`
Lê configurações e listas publicadas.

POST `/alonguebe/v1/appointments`
Cria uma solicitação de agendamento.

POST `/alonguebe/v1/messages`
Cria uma mensagem de contato.

As rotas são públicas porque visitantes não estão conectados ao WordPress. Antes de um lançamento real, é recomendável adicionar CAPTCHA, limitação de requisições e monitoramento contra spam.

10. VITE, DESENVOLVIMENTO E PRODUÇÃO

Durante o desenvolvimento, Vite:

- abre o site em `http://localhost:3000`;
- atualiza a tela rapidamente após alterações;
- transforma JSX para JavaScript entendido pelo navegador;
- encaminha `/wp-json` ao WordPress local;

- mantém /api apontando para o Express apenas por compatibilidade.

O comando principal é:

```
PATH="$PWD/.node-bin/bin:$PATH" npm start
```

Em produção, execute:

```
PATH="$PWD/.node-bin/bin:$PATH" npm --prefix client run build
```

O resultado aparece em client/dist. Essa pasta é gerada e não deve ser editada.

Para produção, VITE_WORDPRESS_URL precisa conter a origem pública do CMS, por exemplo <https://cms.seudominio.com.br>, sem /wp-json no final.

Variáveis VITE_* ficam visíveis no navegador. Nunca coloque senhas nelas.

=====

10.1 EXEMPLOS PRÁTICOS COM CODE SNIPPETS

=====

Os exemplos abaixo foram retirados ou simplificados a partir do projeto. As linhas foram mantidas curtas para facilitar a leitura no PDF.

SNIPPET 1 – UM COMPONENTE REACT SIMPLES

```
function Welcome({ name }) {  
  return <h1>Olá, {name}!</h1>;  
}  
  
<Welcome name="Rafael" />
```

Como ler:

- Welcome é uma função e também um componente.
- { name } retira a prop name do objeto recebido.
- return informa o JSX que aparecerá na tela.
- {name} dentro do JSX insere um valor JavaScript.
- O uso do componente produz “Olá, Rafael!”.

No projeto real, Header, Hero e Footer seguem a mesma ideia, porém retornam estruturas maiores.

SNIPPET 2 – MONTANDO O REACT NO HTML

Trecho simplificado de client/src/main.jsx:

```
ReactDOM.createRoot(  
  document.getElementById('root')  
)  
)  
<React.StrictMode>  
  <CmsProvider>  
    <App />  
  </CmsProvider>  
</React.StrictMode>  
)  
);
```

Como ler:

- `getElementById` procura `<div id="root">` em `index.html`.
- `createRoot` transforma esse elemento na raiz controlada pelo React.
- `CmsProvider` fica ao redor de `App` e compartilha os dados do CMS.
- `App` representa todo o site.
- `StrictMode` ajuda a detectar problemas durante o desenvolvimento.

Padrão reutilizável: quase todo projeto React possui um ponto de entrada parecido. Providers globais devem ficar ao redor do componente principal.

SNIPPET 3 – ESTADO COM `useState`

Trecho simplificado do menu em `Header.jsx`:

```
const [isOpen, setIsOpen] = useState(false);

function toggleMenu() {
  setIsOpen(!isOpen);
}

<button onClick={toggleMenu}>
  Abrir menu
</button>
```

Como ler:

- `isOpen` guarda o valor atual.
- `setIsOpen` é a função autorizada a alterar o valor.
- `false` é o valor inicial: o menu começa fechado.
- `!isOpen` inverte `false` para `true` ou `true` para `false`.
- `onClick` executa `toggleMenu` quando o botão é clicado.

Uma forma ainda mais segura quando o novo valor depende do anterior é:

```
setIsOpen((previous) => !previous);
```

Padrão reutilizável: modal aberto, aba selecionada, aviso visível e filtros também podem ser controlados com `useState`.

SNIPPET 4 – ATUALIZANDO UM FORMULÁRIO CONTROLADO

Trecho usado nos formulários:

```
const [formData, setFormData] = useState({
  name: '',
  email: '',
  message: '',
});

function handleChange(event) {
  const { name, value } = event.target;

  setFormData((previous) => ({
    ...previous,
    [name]: value,
  }));
}
```

Um campo conectado a esse estado:

```
<input
  name="email"
  value={formData.email}
  onChange={handleChange}
/>
```

Como ler:

- `event.target` é o campo que disparou o evento.
- `name` informa qual propriedade deve mudar.
- `value` contém o texto atual do campo.
- `...previous` copia todos os campos existentes.
- `[name]` usa dinamicamente “name”, “email” ou “message”.
- `value` e `onChange` deixam o input controlado pelo React.

Sem `...previous`, a atualização apagaria os outros campos do objeto.

SNIPPET 5 – VALIDAÇÃO ANTES DO ENVIO

Exemplo baseado em `AppointmentForm.jsx`:

```
function validate() {
  const newErrors = {};

  if (!formData.name.trim()) {
    newErrors.name = 'O nome é obrigatório.';
  }

  if (!formData.email.trim()) {
    newErrors.email = 'O e-mail é obrigatório.';
  }

  setErrors(newErrors);

  return Object.keys(newErrors).length === 0;
}
```

Como ler:

- `trim` remove espaços no início e no fim.
- `!` transforma uma string vazia numa condição verdadeira.
- Cada erro é guardado com o nome do respectivo campo.
- `Object.keys` devolve os nomes dos erros existentes.
- Se o tamanho for zero, o formulário pode continuar.

No JSX, uma mensagem aparece somente quando existe:

```
{errors.name && (
  <div className="form-error">
    {errors.name}
  </div>
)}
```

O operador `&&` significa: renderize o segundo trecho somente se o primeiro valor for verdadeiro.

SNIPPET 6 – ENVIANDO JSON COM `fetch`

Exemplo baseado no agendamento:

```
const response = await fetch(
  getCmsApiUrl('appointments'),
  {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(formData),
  }
);

const data = await response.json();

if (!response.ok) {
  throw new Error(data.error || 'Falha no envio.');
```

Como ler:

- await aguarda a conclusão de uma Promise.
- POST indica criação ou envio de dados.
- Content-Type avisa que o corpo usa JSON.
- JSON.stringify transforma o objeto em texto JSON.
- response.json transforma a resposta de volta em objeto.
- response.ok é true para respostas HTTP entre 200 e 299.

fetch não considera HTTP 400 ou 500 uma falha de conexão. Por isso o código precisa conferir response.ok explicitamente.

SNIPPET 7 – try, catch e finally

```
setIsSubmitting(true);

try {
  await sendForm();
  setStatus('success');
} catch (error) {
  console.error(error);
  setStatus('error');
} finally {
  setIsSubmitting(false);
}
```

Como ler:

- try contém a operação que pode falhar.
- catch recebe o erro e permite mostrar feedback.
- finally sempre executa, com sucesso ou erro.
- Por isso finally é o local correto para liberar o botão novamente.

Padrão reutilizável: carregamento de listas, login, upload e qualquer operação assíncrona com estado visual.

SNIPPET 8 – CARREGANDO DADOS COM useEffect

Exemplo simplificado de CmsContext.jsx:

```
useEffect(() => {
  const controller = new AbortController();

  fetch('/wp-json/alonguebe/v1/content', {
    signal: controller.signal,
  })
    .then((response) => response.json())
    .then((content) => {
      setState({
        content,
        loading: false,
        error: null,
      });
    });
}, []);

return () => controller.abort();
}, []);
```

Como ler:

- useEffect executa depois da renderização.
- [] significa que o efeito pertence à montagem do componente.
- AbortController cria uma forma de cancelar a requisição.
- signal conecta o fetch ao controller.
- O return de um efeito é sua função de limpeza.
- abort evita uma atualização tardia quando o componente já saiu da tela.

Em React StrictMode, efeitos podem ser iniciados e limpos mais de uma vez no desenvolvimento. Uma boa função de limpeza torna esse comportamento seguro.

----- SNIPPET 9 – CRIANDO E CONSUMINDO CONTEXT -----

Estrutura simplificada:

```
const CmsContext = createContext(null);

function CmsProvider({ children }) {
  const value = { content, loading, error };

  return (
    <CmsContext.Provider value={value}>
      {children}
    </CmsContext.Provider>
  );
}

function useCms() {
  return useContext(CmsContext);
}
```

Uso num componente:

```
const { content } = useCms();
const title = content?.settings?.hero_title;
```

Como ler:

- createContext cria o canal compartilhado.
- Provider define o valor disponível aos descendentes.
- children representa tudo que foi colocado dentro do Provider.
- useContext lê o valor mais próximo na árvore.
- ?. interrompe o acesso com segurança se algo for null ou undefined.

Use Context para informações realmente globais. Estados usados por apenas um ou dois componentes normalmente ficam melhores em useState e props.

SNIPPET 10 – FALLBACK COM O OPERADOR ||

Trecho semelhante ao usado em App.jsx:

```
const phone =  
  cms.phone || SALON_CONFIG.phoneFormatted;  
  
const title =  
  cms.hero_title ||  
  page?.title?.rendered ||  
  'Alonguebe';
```

Como ler:

- || escolhe o primeiro valor considerado verdadeiro.
- Se cms.phone estiver vazio, usa a configuração local.
- O título possui três níveis de prioridade.
- ?. impede erro caso page ainda não tenha sido carregada.

Atenção: || também troca 0 e false pelo fallback. Quando esses valores forem válidos, prefira o operador ??, que troca apenas null e undefined.

SNIPPET 11 – TRANSFORMANDO UMA LISTA COM map

```
const services = [  
  { id: 1, name: 'Corte' },  
  { id: 2, name: 'Coloração' },  
];  
  
<div className="services-grid">  
  {services.map((service) => (  
    <article key={service.id}>  
      <h3>{service.name}</h3>  
    </article>  
  ))}  
</div>
```

Como ler:

- map visita cada objeto do array.
- Para cada serviço, retorna um article.
- service.id é usado como key estável.
- service.name aparece no título.

Evite usar o índice do array como key quando os itens podem mudar de ordem.

SNIPPET 12 – SERVIÇO SELECIONADO ENTRE COMPONENTES

Em App.jsx:

```
const [selectedService, setSelectedService] =
  useState('');

<Services
  onSelectService={setSelectedService}
/>

<AppointmentForm
  selectedService={selectedService}
/>
```

Em AppointmentForm.jsx:

```
useEffect(() => {
  if (selectedService) {
    setFormData((previous) => ({
      ...previous,
      service: selectedService,
    }));
  }
}, [selectedService]);
```

Como ler:

- App é o dono do estado compartilhado.
- Services recebe uma função capaz de atualizar esse estado.
- AppointmentForm recebe o valor atualizado.
- [selectedService] executa o efeito quando essa prop muda.

Esse é o exemplo real de “elevar o estado” discutido anteriormente.

SNIPPET 13 – URL CONFIGURÁVEL POR AMBIENTE

```
const wordpressUrl = (
  import.meta.env.VITE_WORDPRESS_URL || ''
).replace(/\/$/, '');

const apiUrl = wordpressUrl
  ? `${wordpressUrl}/wp-json`
  : '/wp-json';
```

Como ler:

- import.meta.env contém variáveis expostas pelo Vite.
- replace remove uma barra final para evitar barras duplicadas.
- condição ? valorA : valorB é um operador ternário.
- Em produção usa a URL configurada.
- Em desenvolvimento usa /wp-json e deixa o proxy trabalhar.

SNIPPET 14 – ROTA REST NO WORDPRESS

Exemplo formatado a partir do plugin:

```
register_rest_route(
  'alonguebe/v1',
```

```

    '/content',
    array(
        'methods' => 'GET',
        'callback' => 'alonguebe_content_endpoint',
        'permission_callback' => '__return_true',
    )
);

```

Como ler:

- alonguebe/v1 é o namespace e versão da API.
- /content é o caminho da rota.
- GET significa leitura.
- callback é a função executada ao receber a requisição.
- __return_true permite acesso sem login.

Rotas públicas são necessárias para visitantes, mas precisam de proteção extra quando recebem dados, como CAPTCHA e limitação de requisições.

SNIPPET 15 – VALIDANDO DADOS NO PHP

Exemplo baseado em alonguebe_create_submission:

```

$data = $request->get_json_params();

$name = sanitize_text_field(
    $data['name'] ?? ''
);

$email = sanitize_email(
    $data['email'] ?? ''
);

if (!$name || !is_email($email)) {
    return new WP_Error(
        'invalid_data',
        'Nome e e-mail válido são obrigatórios.',
        array('status' => 400)
    );
}

```

Como ler:

- get_json_params lê o JSON enviado pelo React.
- ?? '' usa string vazia quando a chave não existe.
- sanitize_text_field e sanitize_email higienizam valores.
- is_email confere o formato do e-mail.
- WP_Error devolve uma falha padronizada com HTTP 400.

Higienizar e validar são tarefas diferentes. Higienizar limpa o dado. Validar decide se o dado atende às regras da aplicação.

SNIPPET 16 – CRIANDO UM REGISTRO NO WORDPRESS

```

$post_id = wp_insert_post(array(
    'post_type' => 'along_appointment',
    'post_status' => 'publish',
    'post_title' => $name,

```

```

        'post_content' => $notes,
    ));
    update_post_meta(
        $post_id,
        '_along_status',
        'Pendente'
    );

```

Como ler:

- wp_insert_post cria um item no banco do WordPress.
- post_type decide em qual menu ele aparece.
- post_title facilita identificar o cliente.
- post_content guarda o texto principal.
- update_post_meta salva informação adicional pesquisável.

Depois de wp_insert_post, o código real confere is_wp_error antes de usar o ID.

SNIPPET 17 – PROTEGENDO UM SALVAMENTO ADMINISTRATIVO

```

if (
    !isset($_POST['alonguebe_meta_nonce']) ||
    !wp_verify_nonce(
        $_POST['alonguebe_meta_nonce'],
        'alonguebe_meta'
    ) ||
    !current_user_can('edit_post', $post_id)
) {
    return;
}

```

Como ler:

- isset confere se o campo de segurança chegou.
- wp_verify_nonce confirma a origem do formulário.
- current_user_can confere a permissão do usuário.
- Se qualquer teste falhar, return interrompe o salvamento.

Nonce não substitui permissão. As duas verificações resolvem problemas diferentes.

SNIPPET 18 – VARIÁVEIS CSS

```

:root {
    --primary-rose: #e8c5b8;
    --text-dark: #2c2523;
    --border-radius-md: 12px;
}

.service-card {
    color: var(--text-dark);
    border: 1px solid var(--primary-rose);
    border-radius: var(--border-radius-md);
}

```

Como ler:

- :root declara valores globais.
- --nome cria uma variável CSS.
- var(--nome) reutiliza o valor.
- Uma alteração central pode atualizar muitos componentes.

Esse conjunto de decisões reutilizáveis forma parte de um design system.

SNIPPET 19 – MEDIA QUERY RESPONSIVA

```
.services-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

@media (max-width: 768px) {
  .services-grid {
    grid-template-columns: 1fr;
  }
}
```

Como ler:

- Em telas maiores, o grid possui três colunas iguais.
- max-width aplica regras quando a tela tem até 768 pixels.
- No celular, os cards ficam numa única coluna.

Teste responsividade pelo conteúdo, e não somente por modelos específicos de telefone.

SNIPPET 20 – INICIANDO UM PROCESSO FILHO NO NODE.JS

Exemplo simplificado de scripts/start.js:

```
const child = spawn(
  process.execPath,
  [viteScript],
  {
    cwd: clientDirectory,
    stdio: 'inherit',
    env: process.env,
  }
);
```

Como ler:

- spawn abre outro processo sem bloquear o inicializador.
- process.execPath reutiliza o Node que executou npm start.
- cwd define client como diretório de trabalho.
- stdio: inherit mostra os logs no mesmo terminal.
- env repassa PATH e outras variáveis.

Esse padrão é útil para criar inicializadores de desenvolvimento, mas scripts npm simples devem ser preferidos quando não é preciso controlar sinais.

11. CATÁLOGO DE TODOS OS ARQUIVOS DO PROJETO

Esta seção descreve os arquivos mantidos no projeto. Pastas de dependências e arquivos de build são explicados como grupos porque são gerados automaticamente.

----- RAIZ -----

README.md

Manual operacional. Explica requisitos, instalação, execução, WordPress, variáveis de ambiente, publicação e cuidados de segurança. É o primeiro arquivo que uma pessoa deve ler para conseguir rodar o projeto.

DOCUMENTACAO.md

Guia técnico de manutenção. Explica arquitetura, APIs, estado, formulários, plugin, alterações comuns e conceitos. É mais técnico que o README.

package.json

Manifesto npm da raiz. O script start executa scripts/start.js. dev é um atalho equivalente. start:legacy-api inicia o Express antigo separadamente. share abre um túnel temporário do Cloudflare para a porta 3000.

scripts/start.js

Inicializador do desenvolvimento. Usa child_process.spawn para abrir diretamente o Vite instalado em client/node_modules. Repassa logs e variáveis ao terminal. Ao receber Ctrl+C, envia SIGTERM e, se necessário, SIGKILL depois de dois segundos.

robots.txt

Instruções para robôs de busca. Informa o que pode ser rastreado e normalmente indica a localização do sitemap. Precisa apontar para o domínio final.

sitemap.xml

Mapa de URLs entregue a mecanismos de busca. Como o site atual é uma página única com seções, deve permanecer alinhado ao domínio realmente publicado.

----- FRONTEND -----

client/package.json

Manifesto do frontend. Declara React e ReactDOM como dependências e Vite, plugin React e tipos como ferramentas de desenvolvimento. Contém os comandos dev, build e preview.

client/package-lock.json

Arquivo gerado pelo npm. Registra as versões exatas das dependências para que outras instalações reproduzam o mesmo ambiente. Não deve ser editado à mão.

client/.env.example

Modelo da variável VITE_WORDPRESS_URL. Deve ser copiado para um arquivo de ambiente apropriado na publicação, sem guardar segredos.

client/vite.config.js

Configura o Vite. Ativa o plugin React, usa a porta 3000 e define proxies. O proxy /wp-json aponta ao WordPress local. /api aponta ao Express legado.

client/index.html

Documento HTML inicial. Define idioma, título, descrição, canonical, Open Graph, Twitter Cards, favicon, Google Fonts e JSON-LD de BeautySalon. Contém div#root, onde o React é montado, e importa src/main.jsx.

client/favicon.svg

Ícone vetorial exibido na aba do navegador e nos favoritos. SVG mantém nitidez em tamanhos diferentes.

client/src/main.jsx

Ponto de entrada do React. Importa estilos, envolve App em React.StrictMode e CmsProvider e renderiza tudo dentro de #root.

client/src/App.jsx

Componente raiz. Importa e ordena todas as seções. Guarda fallbacks gerais em SALON_CONFIG, mescla-os com o CMS, controla selectedService, monta links do WhatsApp e envia dados compartilhados por props.

client/src/styles.css

Folha de estilos global. Contém variáveis de cor e tipografia, reset, animações, layouts, cards, botões, formulários, navegação e regras responsivas. As variáveis em :root são o ponto mais seguro para mudar a identidade visual.

----- CONTEXTO, HOOK E SERVIÇO -----

client/src/context/CmsContext.jsx

Busca a rota agregada do plugin uma vez. Guarda content, loading e error e os disponibiliza pelo hook useCms. getCmsApiUrl cria os endereços de formulários. Usa AbortController para cancelar a consulta se o Provider sair da tela.

client/src/hooks/useWordPressPage.js

Hook que busca uma página pelo slug. Controla page, isLoading e error. Em caso de falha, registra um aviso e permite que o componente use conteúdo reserva. Também cancela a requisição ao desmontar.

client/src/services/wordpress.js

Cliente da REST API padrão /wp/v2. getPageBySlug pede somente campos necessários e inclui dados da imagem destacada com _embed. getFeaturedImageUrl extrai a URL da imagem sem obrigar os componentes a conhecerem a estrutura complexa da API.

----- COMPONENTES -----

client/src/components/Header.jsx

Cabeçalho responsivo. Controla abertura do menu móvel e detecta quando a página foi rolada mais de 50 pixels para aplicar o estilo compacto. A lista navLinks gera a navegação por âncoras. Também monta o botão de WhatsApp.

client/src/components/Hero.jsx

Primeira seção. Consulta a página início, usa sua imagem destacada como fundo e combina conteúdo de página, configurações do plugin e fallbacks. Exibe chamadas para WhatsApp e serviços. O HTML editorial é inserido com dangerouslySetInnerHTML.

client/src/components/About.jsx

Seção institucional. Consulta a página sobre, escolhe três imagens do CMS ou de reserva e mostra características do salão. O conteúdo da página WordPress pode substituir os parágrafos locais.

client/src/components/Services.jsx

Catálogo de serviços. Possui uma lista local completa com ícones SVG. Quando o CMS devolve serviços publicados, eles substituem a lista local. Ao clicar num card, comunica o nome a App e rola até o formulário.

client/src/components/AppointmentForm.jsx

Formulário controlado de agendamento. Mantém campos, erros, carregamento, status e cópia do envio bem-sucedido. Valida dados, envia à API WordPress, mostra feedback, limpa os campos e monta uma mensagem formatada para o WhatsApp.

client/src/components/Gallery.jsx

Galeria de trabalhos. Possui itens locais com imagem, categoria, título e texto alternativo. Itens publicados no CMS substituem a lista inteira. loading=lazy

adia imagens fora da tela para melhorar o carregamento inicial.

client/src/components/Testimonials.jsx

Exibe avaliações. Usa depoimentos do CMS ou três exemplos locais. Converte cada nota numa sequência de estrelas e calcula iniciais para o avatar textual.

client/src/components/Location.jsx

Apresenta endereço, telefone, horário e mapa. Cria uma URL para abrir direções e outra para incorporar o Google Maps. Os dados principais chegam de App por props.

client/src/components/FAQ.jsx

Acordeão de perguntas frequentes. O estado `activeIndex` guarda qual pergunta está aberta. Clicar na mesma pergunta novamente fecha o conteúdo. Usa lista do CMS ou perguntas locais.

client/src/components/Contact.jsx

Mostra WhatsApp, Instagram, e-mail, endereço e horários. Controla o formulário de contato e envia seu JSON à rota `messages`. O estado de envio escolhe entre carregamento, sucesso e erro.

client/src/components/Footer.jsx

Rodapé com marca, texto, navegação, links úteis, endereço e redes. Recebe dados por props, consulta textos do CMS e calcula automaticamente o ano atual.

client/src/components/ScrollAnimations.jsx

Componente sem interface própria. Usa `IntersectionObserver` para detectar quando elementos entram na tela e adicionar classes CSS. Respeita `prefers-reduced-motion` para pessoas que preferem menos movimento. Refaz a observação quando chega CMS.

----- ARQUIVOS DE IMAGEM -----

client/public/images/hero-salao-alonguebe.png

Imagem local preparada para a seção Hero. Arquivos em `public` são copiados sem alteração e podem ser acessados a partir da raiz do site.

client/public/images/diferenciais-salao-background.png

Imagem do ambiente usada também como item reserva na galeria.

client/public/images/galeria-alongamento.png

Imagem local de resultado de alongamento usada no fallback da galeria.

client/public/images/galeria-manicure.png

Imagem local de manicure usada no fallback da galeria.

----- PLUGIN WORDPRESS -----

wordpress/alonguebe-headless-cms/alonguebe-headless-cms.php

Plugin completo. Protege execução direta, declara campos gerais, registra CPTs e metadados, cria caixas e colunas administrativas, higieniza salvamentos, monta a tela Alonguebe CMS, transforma posts em dados para o React e registra as três rotas REST. É o backend principal do projeto atual.

----- BACKEND LEGADO -----

server/package.json

Manifesto da API antiga. Declara Express, CORS e `body-parser`. O código atual usa `express.json` diretamente, portanto `body-parser` está instalado mas não é usado.

server/package-lock.json

Versões exatas das dependências do backend legado. Gerado pelo npm.

server/server.js

Servidor Express antigo na porta 5001. Aceita POST /api/appointments, valida os campos e grava em JSON. GET /api/appointments devolve todos os registros sem autenticação. Não deve armazenar dados reais sem revisão de segurança.

server/appointments.json

Array JSON usado como armazenamento pela API antiga. O frontend atual não grava nesse arquivo. JSON não permite comentários.

----- PASTAS GERADAS/OCULTAS -----

.node-bin/

Node.js e npm portáteis para macOS arm64. Permite executar o projeto sem uma instalação global. Não editar manualmente.

.tools/

Ferramentas auxiliares, incluindo cloudflared usado por npm run share.

client/node_modules/ e server/node_modules/

Código das bibliotecas instaladas pelo npm. São recriadas com npm install. Nunca faça manutenção diretamente nelas.

client/dist/

Build de produção gerado pelo Vite. É o resultado que vai para a hospedagem estática. Qualquer alteração direta será perdida no próximo build.

=====

12. COMO FAZER ALTERAÇÕES COM SEGURANÇA

=====

ALTERAR UM TEXTO

Prefira o painel WordPress. Se o texto também precisa aparecer sem CMS, atualize o fallback no componente correspondente.

ALTERAR TELEFONE, ENDEREÇO OU INSTAGRAM

Atualize o painel, SALON_CONFIG e os dados de SEO em index.html. Confira também robots.txt e sitemap.xml quando houver mudança de domínio.

ADICIONAR UM SERVIÇO

Cadastre e publique no WordPress. Para manter o modo offline, adicione o mesmo nome às listas locais de Services e AppointmentForm.

ADICIONAR UM CAMPO GLOBAL AO CMS

Inclua o campo em alonguebe_setting_fields. Depois leia cms.nome_do_campo no componente e forneça um fallback.

ADICIONAR UM CAMPO AO AGENDAMENTO

Atualize o estado, o JSX, a validação e o WhatsApp no React. Depois registre, higienize, salve e apresente o campo no plugin WordPress.

MUDAR CORES OU FONTES

Comece pelas variáveis :root em styles.css. Só depois altere regras individuais. Teste telas grandes e pequenas.

=====

13. SEGURANÇA, ACESSIBILIDADE E QUALIDADE

=====

SEGURANÇA

- Não coloque segredos no frontend.

- Adicione proteção antispam às rotas públicas.
- Faça backup do banco WordPress.
- Dê acesso ao painel somente a pessoas confiáveis.
- Use HTTPS em produção.
- Não exponha o GET do servidor legado com dados pessoais.

ACESSIBILIDADE

- Mantenha textos alternativos úteis nas imagens.
- Preserve labels ligados aos inputs com htmlFor e id.
- Não remova aria-label e aria-expanded do menu.
- Verifique contraste após mudar cores.
- Preserve o tratamento de prefers-reduced-motion.
- Teste navegação usando apenas o teclado.

QUALIDADE

- Execute o build depois de alterações.
- Teste o site com o WordPress ligado e desligado.
- Teste sucesso, erro e carregamento dos dois formulários.
- Confira links externos e números do WhatsApp.
- Não edite lockfiles, node_modules ou dist manualmente.

14. ROTEIRO DE TESTE MANUAL

1. Inicie o WordPress local.
2. Na raiz do projeto, execute npm start com o Node portátil.
3. Abra <http://localhost:3000>.
4. Confira se não há erros no console do navegador.
5. Abra e feche o menu móvel.
6. Clique em cada link de navegação.
7. Escolha um serviço e confirme o preenchimento do formulário.
8. Tente enviar campos vazios e confira os erros.
9. Faça um envio válido e confira o painel Agendamentos.
10. Envie uma mensagem e confira o painel Mensagens.
11. Teste os links de WhatsApp, Instagram e Google Maps.
12. Desligue o WordPress e confira os fallbacks.
13. Execute o build de produção.
14. Verifique a página em largura de celular, tablet e desktop.

15. EXERCÍCIOS PARA ESTUDO

EXERCÍCIO 1 – PROP

Troque temporariamente o número fallback em SALON_CONFIG. Localize quais botões mudam sem editar diretamente os componentes filhos.

EXERCÍCIO 2 – ESTADO

Leia FAQ.jsx e explique, com suas palavras, por que clicar duas vezes na mesma pergunta fecha o conteúdo.

EXERCÍCIO 3 – MAP

Adicione um serviço somente à lista local e identifique como um objeto vira um card e uma option.

EXERCÍCIO 4 – FALLBACK

Com o WordPress desligado, anote quais partes continuam funcionando e quais dependem da API.

EXERCÍCIO 5 – API

No painel WordPress, publique um novo depoimento. Recarregue o site e acompanhe o caminho: banco, plugin, rota content, contexto e componente.

EXERCÍCIO 6 – CSS

Altere uma variável de cor em :root e observe quantos elementos mudam juntos. Desfaça a mudança ao terminar.

16. GLOSSÁRIO RÁPIDO

API: interface usada por programas para trocar dados.
Backend: parte que valida, processa e armazena dados.
Build: versão otimizada criada para publicação.
CMS: sistema de gerenciamento de conteúdo.
Componente: parte reutilizável da interface React.
CORS: regra que controla chamadas entre domínios diferentes.
CPT: tipo personalizado de conteúdo do WordPress.
DOM: representação da página usada pelo navegador.
Endpoint: endereço específico de uma API.
Fallback: valor reserva usado quando o principal falta.
Fetch: função do navegador para realizar requisições HTTP.
Frontend: interface com a qual o visitante interage.
Headless: CMS separado da interface pública.
HTTP 201: resposta indicando criação bem-sucedida.
JSON: formato de texto para transportar dados estruturados.
JSX: sintaxe que combina estrutura visual e JavaScript no React.
Metadado: informação adicional ligada a um conteúdo.
Plugin: extensão que adiciona funções ao WordPress.
Proxy: intermediário que encaminha uma requisição.
REST: estilo comum de construção de APIs HTTP.
Slug: identificador curto e legível usado em URLs.
SPA: aplicação cuja interface é controlada no navegador.
Vite: ferramenta de desenvolvimento e build do frontend.

17. CHECKLIST PARA REUTILIZAR EM UM NOVO PROJETO

- Defina claramente frontend, CMS e responsabilidades.
- Centralize URLs e dados compartilhados.
- Crie fallbacks para conteúdo importante.
- Reutilize contexto para dados globais.
- Separe seções em componentes pequenos.
- Valide formulários no cliente e no servidor.
- Higienize e escape valores no WordPress.
- Planeje autenticação e proteção antispam.
- Use variáveis CSS para o design system.
- Documente arquivos gerados e arquivos manuais.
- Mantenha instruções de instalação reproduzíveis.
- Teste falhas de rede, responsividade e acessibilidade.
- Atualize SEO e dados estruturados antes da publicação.

CONCLUSÃO

O aprendizado principal deste projeto é a separação de responsabilidades. React cuida da experiência visual. O WordPress cuida do conteúdo e dos envios. A API conecta os dois. Context e hooks organizam o acesso aos dados. Componentes dividem a interface em partes compreensíveis. Fallbacks mantêm o conteúdo visível

1
quando o CMS falha.

Ao trabalhar em futuros projetos, não comece copiando todo o código. Comece copiando as ideias: responsabilidades claras, dados centralizados, validação em duas camadas, documentação próxima do código e testes proporcionais ao risco.

FIM DA APOSTILA